

# Congestion Control as a Service: Towards Low Latency Mobile Uploading

Heng Xu\*, Letian Li<sup>†</sup>, Liang Wang\*, Fei Song\*\*

\*School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing, China

<sup>†</sup>Department of Information Engineering, Chinese University of Hong Kong, China

Email: hengxu@bjtu.edu.cn, letianli@cuhk.edu.hk, wangliang1@bjtu.edu.cn, fsong@bjtu.edu.cn

**Abstract**—Ensuring quality of service (QoS) for data transmission in dynamic network environments presents a significant challenge. To solve the conflict between complex network conditions and strict transmission quality, effective solutions employing artificial intelligence have recently been introduced. Nonetheless, the prevalent methods primarily emphasize throughput oriented designs, concentrating on data’s downlink transmission while overlooking the upload of delay sensitive data during bidirectional interactions. Resource-constrained mobile devices struggle not only with executing long-term and continuous online computing tasks but also with rapidly adjusting to the constantly changing wireless networks. Models running on mobile devices must adapt to the changing wireless links. Diverse application demands and limited hardware capabilities pose significant obstacles to deploying relevant models.

Consequently, we introduce a novel congestion control scheme named Colla, which enables mobile devices to achieve on-demand configurations through edge computing services. Colla reduces the computational load on the devices by incurring manageable communication overhead. It offers flexibility to balance latency and throughput according to the application’s requirements, thereby enabling low-latency data uploads. Experimental evaluations using real-world traces across 28 wireless scenarios demonstrate that Colla achieves effective bandwidth utilization, ranging from 66% to 93% in various conditions. Moreover, while maintaining comparable goodput, Colla significantly diminishes delay, reducing the average and the 99.9th percentile delays by 40% and 64%, respectively. Notably, even under conditions of unreliable edge links, Colla outperforms its competitors.

**Index Terms**—Congestion control, Edge computing, Deep reinforcement learning, Quality of service

## I. INTRODUCTION

Both existing real-time live streaming, augmented reality, and promising meta-universes demand stable bidirectional transmission of data in various environments. Thus, reliable low latency communications become a focal point of recent research. It is worth noting that congestion control [1] emerges as a key element in ensuring transmission performance by elaborately controlling the data source rate to circumvent unacceptable packet loss or delay due to resource competition. The primary goal of congestion control was to improve throughput, past methods [2, 3] performed well but recent interactive applications pose new challenges to transmission latency. To improve the real-time performance of the network, simultaneously realizing the goals of low latency and high

throughput has become the mainstay of current congestion control algorithm design [4].

However, it’s observed that the above research and practice predominantly concentrate on downlink data transmission, overlooking the restrictions in uplink transmission, which is critical for interactive applications. Especially in modern wireless systems, traditional congestion control schemes often falter in guaranteeing optimal transmission performance [5]. [6, 7] is tailored for wireless contexts, but their applicability is network-specific. The evolution of adaptive congestion control strategies demonstrates considerable prospects notably those leveraging artificial intelligence, like online learning. These strategies dynamically adjust to network status shifts, aiming to sustain low latency and high data transmission efficiency, thereby fulfilling stringent service demands.

Unfortunately, while these methods excel in particular scenarios, they encounter several limitations. On one hand, modeling computation brings complexity far beyond traditional algorithms, resource-constrained mobile terminals often struggle to manage complex and ongoing online learning tasks [8]. This makes it difficult for the control model to adapt quickly to changing network conditions. In Fig. 1, Transferring the intensive online learning tasks to the edge cloud presents a viable solution. Yet, edge cloud deployment introduces new challenges, including defining the interaction logic between mobile hosts and edge cloud services, ensuring service quality amidst poor communication quality, and managing the communication overhead inherent to these services.

On the other hand, with different services presenting unique QoS demands, designing a model framework that guarantees service generalization and diversity remains a critical problem. For instance, varying business applications necessitate distinct QoS metrics such as throughput, delay, or tail latency. Importantly, current algorithms rarely address how to adapt to varying service demands, making it very challenging to provide the anticipated performance for different applications under dynamic network conditions. A singular congestion control model can only meet service demands to a limited extent and lacks the flexibility to adjust according to these demands. Consequently, there is an urgent need for a novel approach that can reduce computational overhead at the terminal level while also adapting to dynamic environments and user demands.

Based on the above observations, this paper proposes and implements Colla, a novel congestion control scheme utilizing

<sup>1</sup>Fei Song is the corresponding author.

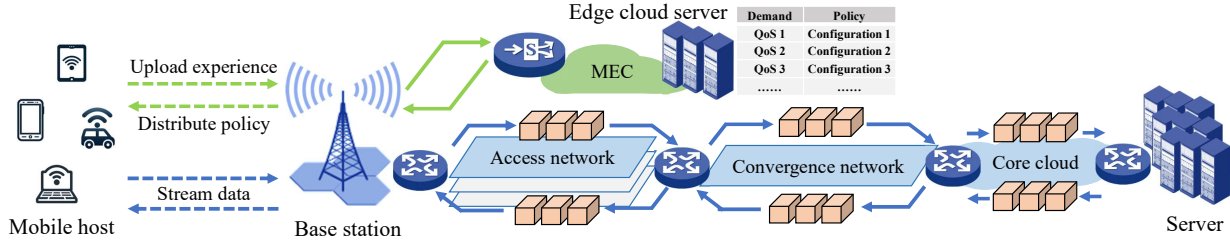


Fig. 1: System overview: Colla scheme for enabling edge cloud server.

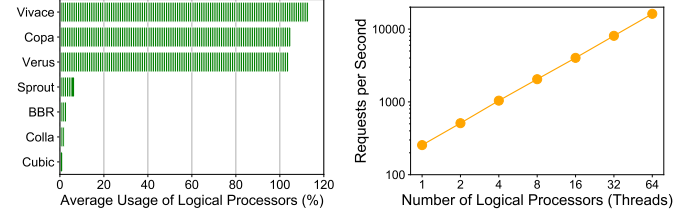
edge computing services. It employs a user-space congestion control strategy similar to CCP [9] through IPC mechanism, achieving minimal kernel-level overhead. By interacting with edge computing services, mobile hosts can secure the optimal congestion control configuration, thereby fostering decision consistency across various endpoints and reducing the burden of online learning. The core objective of this study is to ascertain the ideal balance between throughput demands and latency for diverse applications, and to identify methods for preserving stability and efficiency, even serving with poor communication quality. Furthermore, it delineates the rules for data exchange between mobile hosts and edge cloud services and analyses the associated communication overhead.

To evaluate the performance of the Colla, tests were conducted using real-world traces across 28 wireless scenarios, including bus, taxi, subway, walking, and stationary spots. The conclusion indicates that Colla, in comparison to leading congestion control mechanisms, not only maintains effective throughput utilization (goodput / link capacity) ranging from 66% to 93% in different types of scenarios but also reduces the maximum average delay by 40%, the 95th percentile delay by 49%, the 99th percentile delay by 49%, and the 99.9th percentile delay by 64%. Despite poor communication with edge servers, Colla’s overall performance in tail latency exceeds that of similar solutions, indicating its adaptability and effectiveness in demanding environments.

The contributions of this paper are as follows:

- We have developed a low-latency congestion control scheme Colla enabled by edge services and a modular kernel-based IPC interface allowing for swift construction and the design of a user-space congestion control.
- The format for data exchange and the interaction logic between mobile hosts and edge cloud services are meticulously crafted, alongside an analysis of potential communication overheads.
- Comprehensive testing using real-world network traces reveals that Colla consistently achieves high performance under various edge cloud communication quality in different mobile scenarios.

The structure of this paper is organized as follows: Section II provides the background and motivation; Section III outlines the system design and methodology; Section IV discusses the experiments and evaluation; Section V reviews related work; and the final section offers conclusions and future directions.



(a) The computational resources (b) The concurrency supported by occupied by different schemes. edge cloud servers.

Fig. 2: Local and edge cloud service overhead.

## II. BACKGROUND AND MOTIVATION

The emergence of real-time communication applications has underscored the importance of uplink capabilities. These applications demand rapid data upload and place a significant emphasis on personalization, user experience, and interactivity. With the widespread adoption of wireless networks, accessing real-time applications using small-scale terminals has become the norm. However, in mobile communication environments, link quality frequently fluctuates due to user mobility or environmental interferences. Hence, presently the focus of congestion control has shifted from conventional heuristic algorithms to more adaptive strategies, it is necessary to accommodate varying network conditions as well as to meet diverse performance requirements.

These strategies are valued for their robust expressive capabilities and their aptitude for optimization in intricate network environment [10]. Nonetheless, adaptive methods typically require greater computational resources than traditional algorithms, especially those involving online learning. Given the limited processing capabilities of mobile devices, computationally intensive learning tasks that necessitate prolonged execution may adversely affect device performance, accelerating battery depletion and impacting the utilization of local computing resources for other tasks. One of our design’s objectives is to utilize edge cloud computing services to furnish mobile devices with the best congestion control configurations for current services, without imposing the strain of online learning on the devices themselves. As shown in Fig. 1, Colla offloads computing to the edge cloud.

In Fig. 2(a), we measured the computational overhead of various schemes on a mobile CPU at 3.5GHz. Classic methods like Cubic and BBR leverage kernel efficiencies for minimal CPU utilization. Adaptive methods such as Vivace, Copa, and Verus fully utilize a total logical processor, their

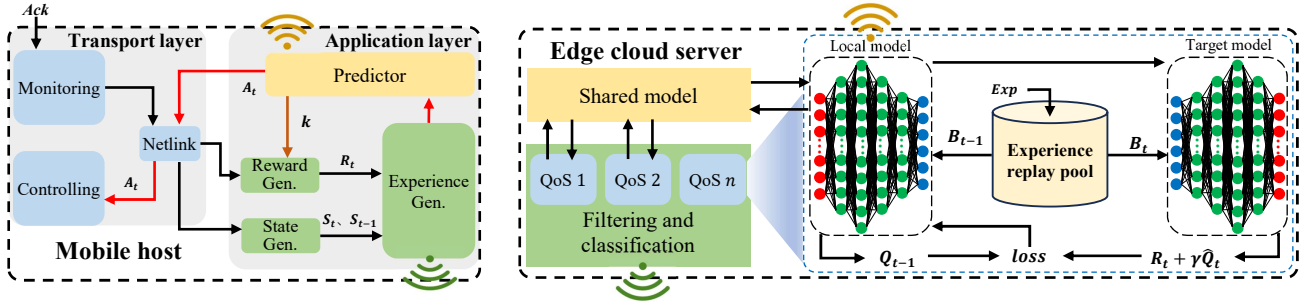


Fig. 3: Architecture details of Colla's scheme.

CPU usage depends on their algorithmic design and iteration strategies. Shorter cycles or complex algorithms raise the usage rate significantly, however, offloading the computation makes local overheads minor. In Fig. 2(b), we measured the computational resources required for various concurrency levels on the server (details in Section IV-A). Due to the lack of dependency between tasks (only gradient uploads), there is a linear relationship between concurrency levels and thread count. With 32 threads, the system manages nearly 8000 concurrent operations, and each iteration lasts approximately 4 ms. Edge servers avoid redundant computations, thus the total computational overhead is lower than on computing on mobile devices.

Furthermore, we have observed that due to the limitations inherent in the layered network architecture, the design of the transport layer often struggles to meet differentiated performance demands. Typically, when transmission is based on TCP, all applications adopt the same congestion control mechanism. However, the presence of large queue buffers prevents the simultaneous optimization of throughput and latency. For latency-sensitive applications, the priority is to ensure minimal end-to-end delay, such as the application layer's GCC [11] used by WebRTC. QUIC [12] considers pluggable congestion control algorithms in its design, aiming to provide the best-suited algorithm on demand for different applications. To the best of our knowledge, there are still no mature practices in this area, and this approach may further increase the computational load on the terminals. Our insight suggests leveraging publicly accessible edge infrastructure to offer on-demand congestion control computation services. Applications with similar transmission requirements could share the access network status of a connection, enabling rapid iteration of models without the necessity for extensive online fitting.

### III. SYSTEM DESIGN

#### A. Architecture Details and Modeling

The IPC interface for mobile hosts can be rapidly developed and deployed through modularization in Fig. 3 on the left, focusing primarily on the Monitoring and Controlling modules alongside the Netlink interface. The user space is chiefly designed with components such as the State, Reward, Experience Generator, and Predictor Module.

**Monitoring:** This module leverages the underlying TCP interface to construct and gather data from acknowledgment

TABLE I: Mainly monitoring information.

| Notation             | Description                                   |
|----------------------|-----------------------------------------------|
| $r_{tt}$             | RTT of last (S)ACKed packet                   |
| $min\_r_{tt}$        | Minimum RTT within a monitoring period        |
| $thr$                | Real time measurement of throughput           |
| $max\_thr$           | Maximum throughput within a monitoring period |
| $cwnd$               | The current congestion window                 |
| $inflight$           | Delivered packages but not acknowledged       |
| $min\_r_{tt\_stamp}$ | Timestamp of the minimum RTT                  |
| $max\_bw\_stamp$     | Timestamp of the maximum throughput           |

(Ack) messages from lower layers. It is tasked with analyzing information to derive statistical data and monitoring various timestamp. For instance, the monitoring of cycle timestamps facilitates the periodic aggregation of parameters assessing network performance. Unlike traditional TCP congestion control mechanisms, which typically adjust the window size based on the number of received Acks—closely tied to the Round-Trip Time (RTT), the monitoring period in learning-based congestion control correlates with RTT and is often set to a multiple thereof [13]. Upon overflow of the monitoring cycle timestamp, statistical data is relayed to the higher layers for the generation of experience samples, detailed in Table I. Here,  $min\_r_{tt}$  and  $max\_thr$  represent the maximum values recorded within two timestamp cycles, respectively, with all additional data being statistical values from the cycle.

**Netlink:** The IPC interface is established via a Netlink Socket. Netlink [14] offers a specialized duplex communication channel between user and kernel processes, perfectly complementing the asynchronous parallel online learning framework. Through Netlink, an efficient data exchange protocol between the kernel's congestion control process and the user process is devised, with each statistic allocated 32 bits.

**Generator:** Utilizing statistical data on network metrics, the Generator constructs local online learning  $experience^{(n-1)} := \{S_{n-1}, A_{n-1}, R_n, S_n\}$  and uploads them through a connectionless socket. Here,  $n$  represents the monitoring cycle rounds. The underlying statistical data  $\{r_{tt}, min\_r_{tt}, thr, max\_thr, cwnd, inflight\}$  are quantized to define the state  $S$ . The state for the previous monitoring cycle  $S_{n-1}$  and the reward  $R_n$ , calculated by the Reward Generator based on this statistical data, facilitate the comparison with the current state  $S_n$  to calculate the action  $A_{n-1}$  relative to the congestion window (cwnd) adjustments.

**Predictor:** Upon uploading experience samples, Predictor

is immediately activated to choose an action based on local predictions. Its configuration is sourced from edge cloud services. The algorithm's ultimate goal is to modulate the cwnd (or pacing rate) over time. The difference of cwnd,  $\Delta cwnd$ , is defined as a discrete integer. In our experiments,  $m$  is set to 8. The action strategy is as follows:

$$\Delta cwnd = \{-m, -m+1, \dots, -1, 0, 1, \dots, m-1, m\} \quad (1)$$

Why is this action space selected? Contrary to download, client's cwnd tend to be limited due to significantly lower upload than download demands. Uploading a 2K/120fps video at 24 bits using H.265 compression, which reduces data size by 300 times, requires 20 Mbps upload bandwidth. Assuming a 30 ms RTT and an upload throughput increase to 40 Mbps, the BDP calculates to 150 KB. Consequently, the congestion window need correspond to this BDP, equaling 103 (with an MSS of 1460 bytes). Even with a window adjustment time of twice the RTT, Colla efficiently utilizes the full upload bandwidth within one second.

**Controlling:** With the TCP protocol operational, the Controlling module adjusts the cwnd in real-time based on the Predictor's output for  $\Delta cwnd$ . If pacing is required, the underlying TCP's pacing module can be initiated to convert the cwnd adjustments into a pacing rate. Prior to the Predictor control, the Controlling module will employ a utility-based slow-start algorithm for initial cwnd management.

**Reward:** Intelligent congestion control schemes leverage utility functions to gauge reward mechanisms. These rewards evaluate the real-time behaviour of the current cwnd within the environment, aiding in the refinement of action strategies. Designed with latency and throughput considerations, reward functions aim to achieve the dual objectives of minimizing latency and maximizing throughput in the network. Packet loss (typically a result of buffer overflow or link failure, particularly in cases of poor wireless link quality) is addressed through a heuristic penalty strategy for persistent losses, rather than continuous learning via reward mechanisms.

In practical network systems, achieving maximum throughput and minimum delay simultaneously is challenging. Due to the inability to fully synchronize various devices on the Internet, striving for bandwidth utilization near 100% on bottleneck links can lead to significant delay spikes. While some algorithms focused on maximizing bandwidth utilization avoid completely filling the buffer, they still increase delay. An alternative approach involves sacrificing a portion of bandwidth to achieve the lowest possible latency, which is the preferred strategy while ensuring the best user experience.

To accommodate diverse custom scenarios, it is prudent to establish a threshold that balances the trade-off between delay and throughput, catering to varying QoS requirements. In the bottleneck link model [3], minimum delay and maximum throughput are assessed in real-time successively to ascertain the flow state. For bandwidth-limited flows, speeds are increased to probe for bandwidth, whereas for application-limited flows, speeds are reduced to minimize latency, thereby swiftly converging to an optimal operational point. In the state

generator, metrics  $\{rtt, min\_rtt, thr, max\_thr\}$  are considered. When throughput variation is minimal over consecutive rounds, if  $rtt \leq k \times min\_rtt$ , the flow is deemed application-limited; if  $rtt > k \times min\_rtt$ , it is considered bandwidth-limited.  $k$  is an identification provided by the edge cloud server to users to distinguish different QoS, which on the one hand affects the generation of rewards in experience samples, and on the other hand correctly uploads experience to specified QoS components. The experience contains user feedback on latency and throughput information, which is beneficial for edge cloud server to adaptively provide different services for mobile hosts. Different states of flows evaluated by distinct reward functions. When the flow is application-limited, the reward function  $R_n$  is defined for throughput improvement:

$$R_n = \frac{thr_n - thr_{n-1}}{|thr_n - thr_{n-1}|} \times \alpha \quad (2)$$

Otherwise, for bandwidth-limited flows, the reward function  $R_n$  accounts for both throughput and delay:

$$R_n = \frac{rtt_{n-1} - rtt_n}{|rtt_{n-1} - rtt_n|} \times \left( \frac{rtt_n - min\_rtt_n}{\delta_2} \times \beta + 0.1 \times \alpha \right) \quad (3)$$

The expressions  $(thr_n - thr_{n-1})/(|thr_n - thr_{n-1}|)$  and  $(rtt_{n-1} - rtt_n)/(|rtt_{n-1} - rtt_n|)$  determine the direction of the reward, positive or negative. The scaling factors  $\delta_1$  and  $\delta_2$  normalize throughput ( $thr, max\_thr$ ) and RTT ( $rtt, min\_rtt$ ) statistics. Here, Constants  $\alpha$  and  $\beta$  serve for normalization:

$$\alpha = \begin{cases} 4x^3 & , \text{ if } x < 0.5 \\ 1 - 16(1-x)^5 & , \text{ otherwise} \end{cases} \quad (4)$$

$$\beta = \begin{cases} 1 - 16x^5 & , \text{ if } x < 0.5 \\ x & , \text{ otherwise} \end{cases} \quad (5)$$

$$x = \min \left( 1, \max \left( 0, \frac{thr_n / \delta_1}{C_{btlbw}} \right) \right) \quad (6)$$

$x \in [0, 1]$  is scaled by the real time throughput over a bottleneck bandwidth constant  $C_{btlbw}$  (where  $C_{btlbw} \geq max\_thr / \delta_1 \geq thr_n / \delta_1$ ), indicating the relative throughput magnitude of the flow. When the bandwidth capacity of the bottleneck link is not fully utilized, flows with lower throughput are provided with increased rewards, affording them enhanced opportunities to acquire additional bandwidth. Conversely, as the buffer begins to bloat, leading to potential congestion, flows with higher throughput incur greater penalties. This strategic adjustment ensures they abandon a portion of the bandwidth, thereby mitigating congestion risks. This mechanism promotes equitable bandwidth distribution among various data flows, ensuring a fair allocation of resources.

In the realm of distributed machine learning, asynchronous deep reinforcement learning (DRL) techniques [15] permit parallel execution of multiple agents across varied network settings. Each agent independently navigates the environment and contributes asynchronous updates to a global model. This methodology aligns well with providing tailored QoS through edge cloud services, where the random access mechanism for

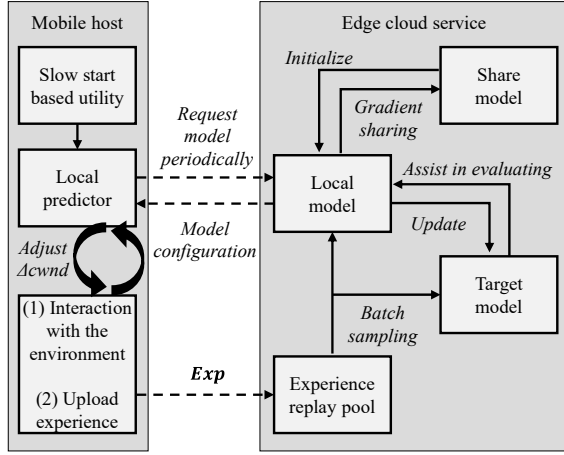


Fig. 4: The interaction logic between mobile host and edge cloud services.

users complements asynchronous learning. By associating different agents with specific QoS components, individual agents engage in targeted learning under distinct network conditions. Consequently, edge cloud services can offer optimal model configurations to meet diverse throughput, delay, and tail latency needs, thus ensuring an exceptional user experience and service variety. In Fig. 3 on the right, the edge cloud service includes a shared model and different QoS components, each housing two models and an experience replay pool.

**Filtering:** Upon the upload of experiences, corresponding  $k$  values and quantified  $min\_rtt$  to direct samples into appropriate QoS components, aligning with their needs. Lost experiences are treated as data loss, requiring no further action.

**Experience Replay Pool:** It stores training experiences, selecting batches randomly for each round to enhance learning efficiency through historical data reuse. It addresses data scarcity and sampling inefficiency, stabilizing the training process for better algorithm convergence and generalization.

**Neural Networks:** Each QoS component operates with a local and a target model, alongside a shared model maintained by the entire edge cloud service. These models, structured as convolutional neural networks, share an identical architecture comprising an input layer, four fully connected layers, and an output layer. The number of neurons in the input layer is the dimension of state  $S$ , with neuron counts in the fully connected layers being 32, 64, 128, and 64, respectively. The output layer's neuron count matches the dimension of action space. Differentiation in model configurations across QoS components enables service diversification, with the shared network ensuring consistent high performance.

## B. Control Logic and Methodology

Congestion control of mobile hosts, online learning process of edge cloud service, and their specific interaction logic are depicted in Fig. 4. Mobile hosts periodically upload samples of their environment-experienced data and obtain optimal model configurations from the edge cloud service.

**Local Congestion Control:** Traditional congestion control relies on heuristic algorithms that initiate backoff algorithms upon detecting potential events to avoid congestion, rather than aiming to pursue performance. This process typically unfolds in four stages: slow start, congestion avoidance, fast retransmission, and fast recovery. In contrast, adaptive schemes seeks to maximize long-term benefits to sustain high performance. Within the Colla framework, each mobile host undertakes two primary functions: congestion control and experience upload.

Upon connection establishment, the mobile host immediately requests model configuration from the edge cloud service. Concurrently, a utility-based slow-start algorithm is activated. If the service request fail prior to completing the slow start, the system reverts to the traditional algorithm. Conversely, a successful startup allows the Predictor to takeover congestion control. The utility-based slow-start algorithm employs a utility function that balances throughput against delay to efficiently requisition link capacity.  $thr'(t) = d(thr)/dt = \lim_{\Delta t \rightarrow 0} (thr(t + \Delta t) - thr(t))/\Delta t$  describes the rate of change in throughput over time.  $thr'(t) \times \Delta t / thr_t$  is used to evaluate the relative degree of throughput change within a given time period. Similarly, the rate of RTT change over time is expressed as  $r_{tt}'(t) = d(r_{tt})/dt = \lim_{\Delta t \rightarrow 0} (r_{tt}(t + \Delta t) - r_{tt}(t))/\Delta t$ . And  $r_{tt}'(t) \times \Delta t / r_{tt}_t$  is used to evaluate the relative degree of change in RTT within a given time. The utility function for the slow start is as follows:

$$utility(t) = \frac{1 + (thr'(t) \times \Delta t) / thr_t}{1 + (r_{tt}'(t) \times \Delta t) / r_{tt}_t} \quad (7)$$

Considering the statistical nature of the measurements over time, the equation for throughput change is approximated through differencing as follows:  $thr'(t) \approx \Delta thr / \Delta t = (thr_{t+\Delta t} - thr_t) / \Delta t$ , and it is similar for RTT. This simplification facilitates calculating the utility function:

$$utility(n) = \frac{thr_n - thr_{n-1}}{r_{tt}_n - r_{tt}_{n-1}} \quad (8)$$

During the utility-based slow start, it is necessary to double the cwnd with each monitoring cycle. To shape traffic, the cycle is segmented into equal time slots, allowing for a gradual increase in the window size per slot. Exiting the slow start occurs when the utility measured across three consecutive time rounds falls below 1. After slow start, the predictor is charged with two main functions: periodically requesting the optimal configuration from edge cloud and executing prediction tasks locally to adjust cwnd with latest strategy. The continuous interaction between mobile hosts and the environment generates new statistical data and experience samples.

**Edge Cloud Services:** Different edge cloud services maintain separate experience reply pools, collecting data from various network bottlenecks to improve adaptability and generalization. In DRL tasks, the QoS component encourages continuous interaction between the mobile host and the network environment, aiming to maximize rewards. Initially, the mobile host's actions are determined based on its current state  $s_n \in S$  and the optimal policy  $\pi : S \rightarrow A$ , computing a potential action  $a_n \in A$ . The mobile host executes  $a_n$  to explore the network environment, generates rewards  $r_n = r(s_n, a_n)$ , and



---

**Algorithm 1** Edge Cloud Service.

---

```

1: procedure MAIN
2:   Initialize shared network  $Q^{(s)}$  weights  $\theta^{(s)}$ , and  $I$  QoS servers
3:   for each  $QoS_i$  server in  $I$  do
4:     Initialize experience replay pool  $P_i$ , update cycle  $k$ 
5:     Initialize local network  $Q_i$  weights from shared network  $\theta \leftarrow \theta^{(s)}$ 
6:     Initialize target network  $\hat{Q}_i$  weights  $\theta^- \leftarrow \theta$ , gradients  $d\theta \leftarrow 0$ 
7:   end for
8:   parallel for each  $QoS_i$  server in  $I$ , at the step  $n$  do
9:     spawn receiveExp( $i, n$ )
10:    spawn distributeModel( $i$ )
11:   end for
12:   sync
13: end procedure

14: procedure RECEIVEEXP( $i, n$ )
15:   while do
16:     Receive new experience  $(s_n, a_n, r_n, s'_n)$  and store it to  $P_i$ 
17:     Sample minibatch of experiences from  $P_i$  randomly
18:      $y = \begin{cases} r_n & \text{for terminal } s' \\ r_n + \gamma \max_{a'} \hat{Q}_i(s', a', \theta^-) & \text{for non-terminal } s' \end{cases}$ 
19:     Gradient zeroing, backpropagation  $d\theta = \frac{\partial (y - Q_i(s, a, \theta))^2}{\partial \theta}$ 
20:     Sharing gradients  $d\theta$  to  $Q^{(s)}$ , and the gradient descent
21:     if  $T \bmod k == 0$  then
22:       Update the target network  $\hat{Q}_i = Q_i$ 
23:     end if
24:   end while
25: end procedure

26: procedure DISTRIBUTEModel( $i$ )
27:   while do
28:     if request then
29:       Distribute local model of  $QoS_i$ 
30:     end if
31:   end while
32: end procedure

```

---

transitions to the next state  $s_{n+1} \in S$ . By repeating this process, the mobile host eventually converges to a stable state. Return is the discounted sum of rewards, aiming to find the optimal policy  $\pi$  for the QoS agent to maximizes:

$$R_n = \sum_{k=0}^K \gamma^k r_{n+k} \quad (9)$$

Action-value function  $Q_\pi(s_n, a_n)$  evaluates the expected return from taking a specific action  $a_n$  in a given state  $s_n$ . In model-free DRL, function approximation, often through neural networks  $Q(s, a; \theta)$  is used to estimate  $Q_\pi$ . Edge cloud services maintain three models: each QoS agent has a local and a target model, plus a shared model for global policies, aiding in maximizing expected return and ensuring service reliability.

Prior to launching a new QoS agent, the shared model's parameters are utilized for the initialization of the corresponding QoS agent. For each type of QoS agent, the target model will predict the maximum  $Q$  value for all possible actions in the next state  $s'$ . And  $r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})$  is used to assist in stable fitting of  $\theta$ . The local model evaluates  $Q(s, a; \theta_i)$  for the current state-action pair to refine the policy  $\pi_\theta$ , effectively serving the associated mobile host. The loss function and trick gradient iterative method is as follows:

$$L_i(\theta_i) = \mathbb{E}(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i))^2 \quad (10)$$

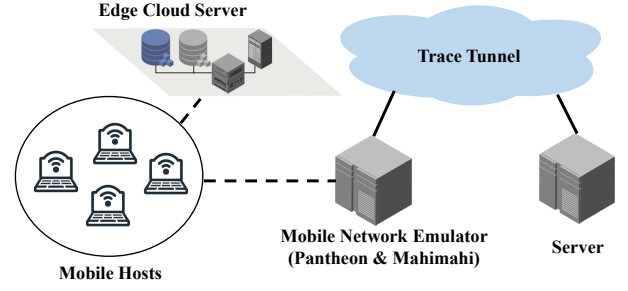


Fig. 5: Trace-driven evaluation topology.

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)) \nabla_{\theta_i} Q(s, a; \theta_i)] \quad (11)$$

The loss function is minimized iteratively using the stochastic gradient descent. Parameters from the local model are regularly synchronized with the target model to enhance the learning process's stability. Each QoS agent independently adjusts its parameters based on its specific experiences. These modifications are periodically integrated into the shared model, with comprehensive details provided in Algorithm 1.

### C. Communication Overhead Analysis

We evaluate Colla's communication overhead for offloading computation to the edge. For uplink, we evaluate Colla's communication overhead for offloading computation to the edge., a UDP message carrying experience data totals 72 bytes ( $8 + 4 \times (7 \times 2 + 1 + 1)$ ), leading to a uplink bandwidth of 7.2kbps per mobile host (Accumulate one experience sample every 80ms). Downlink involves model configuration requests from the QoS agent, with model files sized at 81KB. Initial request intervals of 5s and 10s extend to 20s thereafter, resulting in a first minute downlink need of 45.9kbps per host, which adjusts to 32.4kbps subsequently.

## IV. TRACE-BASED EMULATIONS

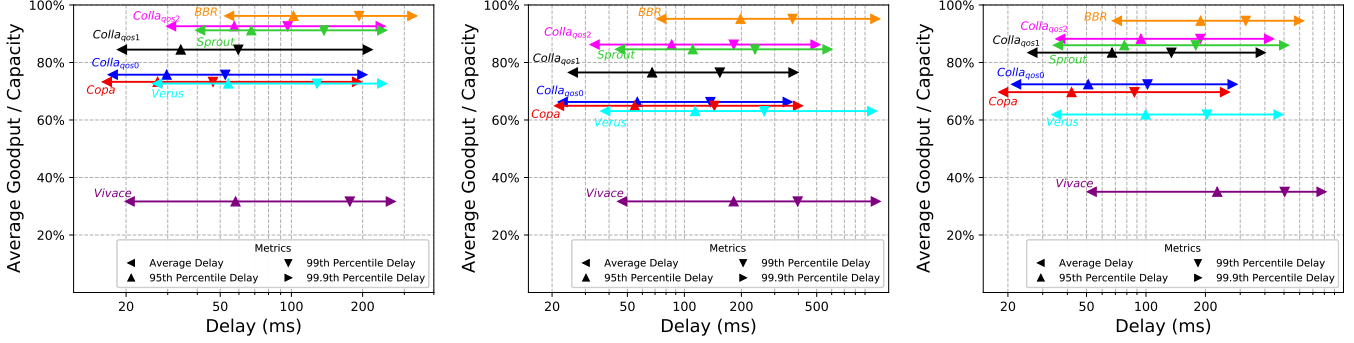
### A. Setup

**Emulator:** The *Colla* congestion control scheme was implemented on a Linux platform, utilizing kernel 5.4. As shown in Fig. 5, to ensure the test results accurately reflect performance in real world application scenarios, multiple schemes was evaluated using the Pantheon [16] verification platform, which is specifically tuned for the Mahimahi [17], facilitating the transmission of actual messages via the TUN/TAP interface on Linux. An Edge Cloud service was deployed, powered by a server equipped with dual AMD EPYC 7642 processors (2.3GHz) that offer a total of 96 cores and 192 threads.

**Setting:** The experimental configuration involved setting the propagation delay between the mobile host and the server to 10ms, the buffer size to 200KB, and the link's random packet loss rate to 1%. The testing encompassed 28 distinct wireless scenarios, including bus, taxi, subway, walking, and stationary spots. The specific information of these real world traces comes from previous works [8, 17, 18]. The scenarios are mainly divided into three categories: home scenario, walking scenario, and vehicle scenario. The default minimum RTT

TABLE II: The Overall Performance Metrics of Multiple Schemes Relative to Colla in 28 wireless scenarios.

| Scheme                  | $Colla_{qos0}$ | $Colla_{qos1}$ | $Colla_{qos2}$ | $BBR$  | $Sprout$ | $Copa$ | $Verus$ | $Vivace$ |
|-------------------------|----------------|----------------|----------------|--------|----------|--------|---------|----------|
| Average ratio           |                |                |                |        |          |        |         |          |
| throughput              | 0.8611         | 1.0000         | 1.1095         | 1.2184 | 1.1367   | 0.8513 | 0.8662  | 0.4215   |
| loss                    | 0.9897         | 1.0000         | 1.0574         | 1.5275 | 5.1053   | 0.9995 | 7.3563  | 4.0191   |
| goodput                 | 0.8611         | 1.0000         | 1.1089         | 1.2109 | 1.0855   | 0.8513 | 0.8085  | 0.4072   |
| delay                   | 0.8496         | 1.0000         | 1.4325         | 2.8261 | 1.7929   | 0.8068 | 1.4225  | 1.4830   |
| 95th percentile delay   | 0.7892         | 1.0000         | 1.4903         | 2.9809 | 1.6766   | 0.7449 | 1.5456  | 2.1295   |
| 99th percentile delay   | 0.8089         | 1.0000         | 1.4354         | 2.8448 | 1.6354   | 0.7665 | 1.6847  | 2.7911   |
| 99.9th percentile delay | 0.9088         | 1.0000         | 1.3553         | 2.9258 | 1.3539   | 0.9437 | 2.5400  | 3.0147   |



(a) Scenarios: in stationary position (home). (b) Scenarios: walking in crowded places. (c) Scenarios: bus, taxi, or subway.

Fig. 6: Average goodput / capacity, delay, 95th, 99th, and 99.9th percentile one-way delay in three types of scenarios.

between the mobile host and the edge cloud server was set to 20ms, with a random fluctuation of  $\pm 2$ ms, and a consistent link random packet loss rate of 1%.

**Schemes Compared:** The *Colla* algorithm was benchmarked against multiple schemes, including delay-sensitive adaptive approaches such as *Sprout* [6], *Verus* [7], and *Copa* [19], the throughput-oriented *BBR* [3], and the learning-based *Vivace* [13] protocol. Given the high packet loss environment (1%) of our test scenarios, the *Cubic* [2] scheme was excluded from comparison due to its expected underperformance in achieving high throughput (reduced by an order of magnitude). We only studied the fairness of *Cubic* on the lossless link. The *Colla* scheme accepts three types of QoS, which we describe as  $Colla_{qos0}$ ,  $Colla_{qos1}$ ,  $Colla_{qos2}$ , with corresponding service flags k of 1.35, 1.5, and 2.0 respectively. Performance metrics can be found in Table II.

### B. General Performance

We directly evaluated the total general performance of multiple schemes in 28 wireless scenarios, using  $Colla_{qos1}$  as the benchmark. Table II describes the overall performance of various schemes relative to *Colla* in 28 wireless scenarios.

**Delay and Tail latency Performance:** In table II, *Colla* exhibits superior performance. From the perspective of high goodput QoS services, compared to *Sprout*,  $Colla_{qos2}$  shows a slight reduction in throughput by about 2.4% and significantly improves latency across various metrics: the average delay is reduced by approximately 20%, the 95th and 99th percentile delays by around 11% and 12% respectively, while the 99.9th percentile delay is almost equal. From the perspective of lower delay QoS services, the performance of  $Colla_{qos0}$  is similar to *Copa* and far superior to *Verus*.  $Colla_{qos0}$  achieves

approximately 6.5% higher goodput, 40% lower average delay, and exhibits reductions in the 95th, 99th, and 99.9th percentile delays by roughly 49%, 52%, and 64% respectively when compared to *Verus*. Fig. 6 shows the average delay, and 95th, 99th, and 99.9th percentile delay in stationary home, crowded walk, and mobile vehicle scenarios. Compared with other algorithms, *Colla* maintains consistent low latency in various scenarios while maintaining the same goodput.

**Goodput Performance:** Fig. 6 shows the average goodput / link capacity corresponding to various schemes in stationary home, crowded walk, and mobile vehicle scenarios. Among the three scenarios, *Colla* has the best goodput performance in the home scenario, followed by vehicle, and the worst in crowded walk scenarios. In the stationary home scenarios, the average goodput / capacity of  $Colla_{qos2}$  reaches 92.6%, whose performance is relatively close to *BBR*, while the *Colla* scheme maintains the lowest delay. Compared with the home scenario, the average goodput / capacity of the *Colla* scheme that receives different three services in the walking scenario decreased by 9.5%, 8.0%, and 6.4%, respectively. While in the mobile vehicle scenario, it decreased by 3.4%, 1.1%, and 4.4%, respectively. In different scenarios,  $Colla_{qos2}$  slightly surpasses the goodput of *Sprout* while maintaining better latency performance. Under any QoS service, the *Colla* scheme outperforms *Verus* in all aspects of performance.

**Packet Loss Analysis:** *Colla* and *Copa* demonstrates the lowest packet rate, with *BBR* experiencing 53% higher packet loss. *Vivace*, *Sprout*, and *Verus* exhibit significantly higher packet losses compared to *Colla*, with increases of nearly 3 times, 4 times, and over 6 times respectively. Regardless of QoS services, *Colla* maintains very similar packet loss rates, performing best in wireless networks.

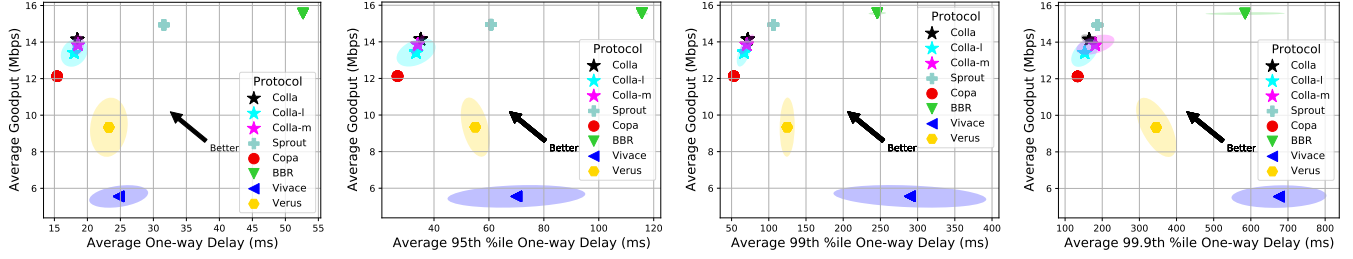
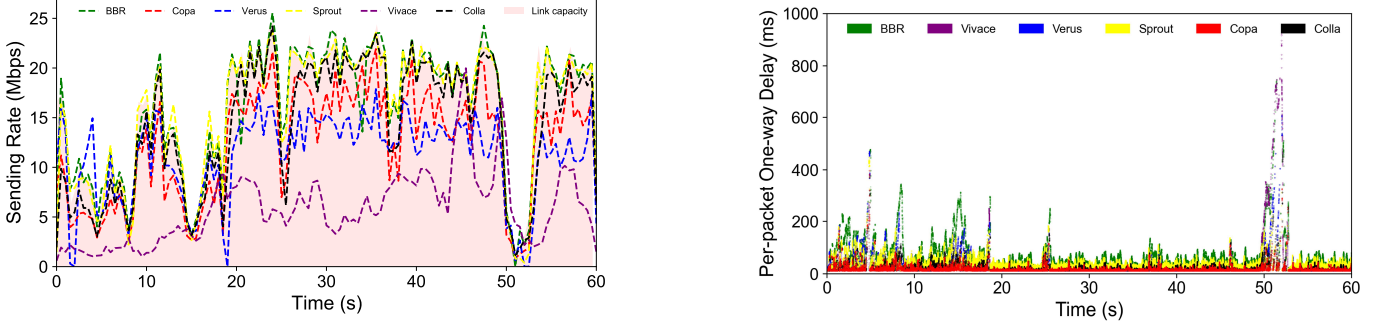


Fig. 7: The average goodput, average delay, 95th, 99th, and 99.9th percentile delay in a bus scenario for different schemes.



(a) Uplink sending rates.

(b) Per-packet one-way delay.

Fig. 8: The uplink sending rates of different schemes and the one way-delay of each packet.

### C. Performance Facing Damaged Services

To explore the effect of varying QoS levels of edge cloud services on the *Colla* scheme's performance, an assessment was conducted within a mobile bus scenario. The benchmark scheme "Colla" is  $Colla_{qos1}$ , featuring default minimum RTT of 20ms with  $\pm 2$ ms random fluctuation and 1% random packet loss rate on the link. The variant "Colla-l" maintains the delay setting of "Colla" ( $Colla_{qos1}$ ) but incorporates four distinct random packet loss rates: 5%, 10%, 15%, and 20%. Conversely, "Colla-m" retains the original packet loss environment of "Colla" ( $Colla_{qos1}$ ), while adjusting the minimum RTT to 40ms, 80ms, 160ms, 320ms, and 640ms with  $\pm 10\%$  jitter.

Fig. 7 shows the average, 95th, 99th, and 99.9th percentile delays and the corresponding average goodput in a bus scenario. Notably, *Colla* demonstrates superior tail latency performance across all conditions, maintaining close proximity to *Sprout* in goodput. The confidence ellipse indicates that *Colla*'s performance remains stable under different service conditions and exhibits only slight fluctuations under impaired service, yet the average performance closely parallels the benchmark "Colla". It can be seen that the mobile hosts using *Colla* can calmly cope with various situations of poor communication with edge computing nodes. It also proves that services can also be provided by the cloud rather than edge.

Fig. 8(a) shows the link capacity and uplink rates over one minute, with *Colla* nearly maximizing link capacity and closely aligning sending rates. Fig. 8(b) displays the one-way delay for packets from various schemes, highlighting *Colla*'s minimal delay. Initially, the poor wireless link quality causes capacity fluctuations. However, *Colla* maintains minimal delay fluctuations compared to others. At the 50th second, facing

near link disconnection, *Colla* adeptly lowers its rate, ensuring exceptional delay performance.

### D. Diversified QoS Services

Currently, video streaming, especially live broadcasts, constitutes the majority of network traffic, necessitating a QoS focus on minimizing latency rather than maximizing throughput. We directly evaluated the performance of  $Colla_{qos0}$ ,  $Colla_{qos1}$ , and  $Colla_{qos2}$  receiving three different types of QoS services in 28 wireless scenarios. In Fig. 9, with the increase of average goodput, various latency indicators of  $Colla_{qos0}$ ,  $Colla_{qos1}$ , and  $Colla_{qos2}$  in any scenario also increase. From this, it can be seen that the *Colla* scheme can balance goodput and delay, ensuring diverse QoS requirements. In the three type scenarios of walk, vehicle, and home, the maximum increment of goodput / capacity of *Colla* scheme is 20%, 21%, and 17%, respectively, and the average delay sacrifices 11ms, 14ms, and 14ms. When considering the tail latency performance, the 95th percentile delay sacrifices 19ms, 43ms, and 28ms. The 99th percentile delay sacrifices up to 4ms5, 87ms, and 44ms. The 99.9th percentile delay sacrifices 141ms, 145ms, 39ms. In Fig. 6, it can be seen that the goodput performance of  $Colla_{qos0}$  is close to *Copa*, and  $Colla_{qos2}$  is close to *Sprout*. When goodput demand rises, regardless of the scenario, the *Colla* scheme is cost-effective, trading minimal latency for high goodput. In summary, *Colla* offers a balanced trade-off between latency and goodput across applications, showcasing diverse performance.

### E. Flow Fairness

In Fig. 10, a new competitive flow joins the link every 30 seconds. In the long term, all schemes are considered



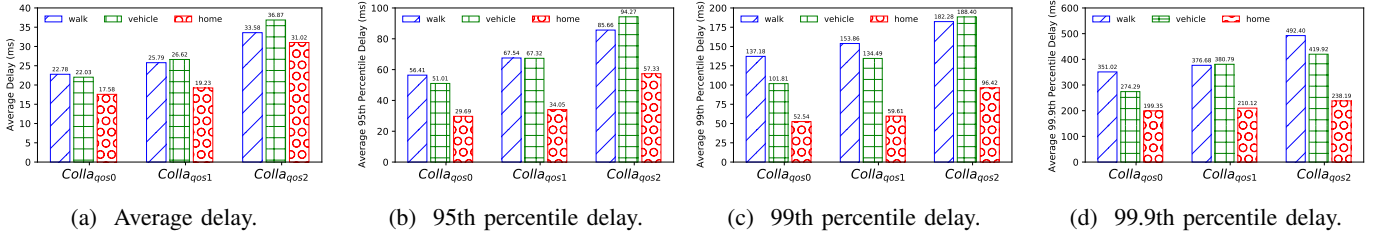


Fig. 9: Performance evaluation of *Colla* schemes under three different types of QoS in 28 mobile scenarios.

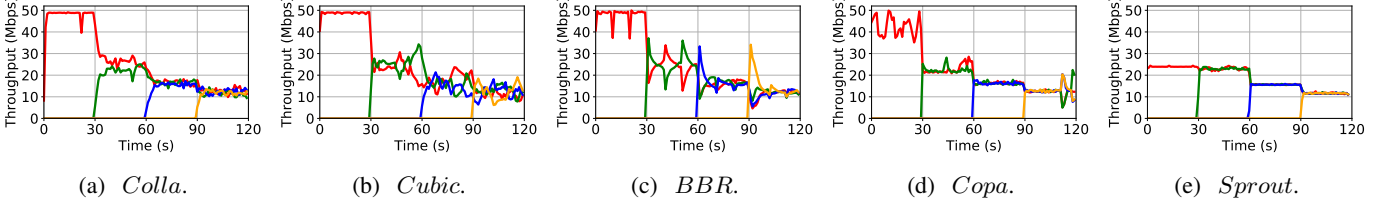


Fig. 10: Every 30 seconds, a new flow joins the competition.

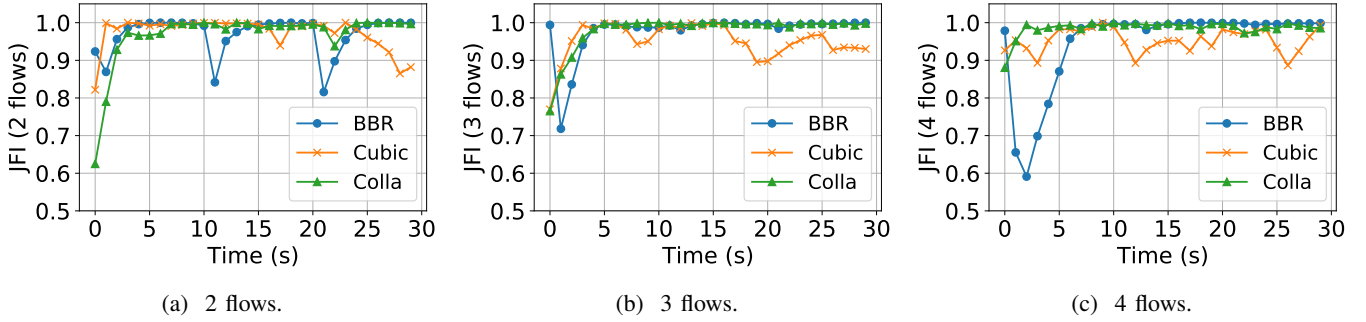


Fig. 11: Jain Fairness Index for short-term time windows (1 second).

relatively fair. For real-time applications, our focus was on assessing short-term fairness. The experimental setup involved establishing RTT of 20ms between the mobile host and the server, configuring the buffer size to 200KB, maintaining a constant link capacity of 48Mbps, and setting the random packet loss rate to 0%. The Jain Fairness Index (JFI) serves as the primary metric for evaluating multi-flow fairness. This index is calculated using the formula:

$$f(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2} \quad (12)$$

where  $x_i$  represents the throughput of the  $i^{th}$  flow, and  $n$  is the total number of flows. The JFI ranges from  $1/n$  to 1, with  $1/n$  indicating the worst fairness and 1 representing perfect fairness. We set the time window to 1 second. Fig. 11 shows the short-term fairness of *Colla* compared to the most classic schemes *Cubic* and *BBR*. When new flows compete, *Colla*'s fairness quickly converges to the highest and remains stable for a long time. On the other hand, *BBR* remains stable for a long time, but as the flow increases, the convergence time increases. Although *Cubic* converges quickly, some short-term fairness may fluctuate.

## V. RELATED WORK

**General Congestion Control:** TCP Tahoe and Reno [1] introduced key AIMD-based congestion control mechanisms.

NewReno [20] enhanced the fast recovery process to reduce timeouts and retransmissions. Bic [21] and Cubic [2] developed cwnd control functions that adapt based on packet loss feedback, indicating network congestion. Vegas [22] utilizes varying RTT to estimate congestion, enabling smoother bandwidth management. Compound [23] combines a latency window with the standard TCP cwnd, improving bandwidth use in high-speed networks. Illinois [24] optimizes window control by monitoring packet loss and latency for long-fat networks. Copa [19] uses utility functions to balance delay with throughput, adjusting pacing rates.

**Scenario-oriented Congestion Control:** Specific environments dictate modern congestion control strategies. DCTCP [4] in data centers uses ECN [25] feedback to modulate the sending window. DCQCN [26] implements rate control on RDMA NICs for fast response in lossless networks. BBR [3] assesses congestion by comparing inflight data with the bandwidth delay product, particularly in B4 data center networks. Sprout [6] and Verus [7] adapt to mobile and variable wireless conditions through latency-sensitive adjustments. C2TCP [27] enhances classical TCP for cellular networks.

**Learning-based Congestion Control:** Learning-based approaches adapt congestion control to diverse network settings. Remy [28] pioneered machine learning for rate adjustment. PCC Allegro [5] and PCC Vivace [13] utilize performance-

driven methods with a focus on long-term fairness and stability. QTCP [29] applies q-learning to manage cwnd, though it incurs high overhead. Indigo [16] and Aurora [30] explore DRL for nuanced rate control, addressing fairness and security. Orca [8] and DeepCC [18] blend classical and DRL approaches to enhance TCP performance in diverse scenarios.

## VI. CONCLUSION

The artificial intelligence-based congestion control algorithm demonstrates strong performance capabilities. Nevertheless, its practical deployment encounters challenges due to the computational overhead and hardware constraints. Our proposed solution, Colla, leverages edge computing to offer congestion control services tailored to mobile terminals. Colla dynamically provides computing models suited to various applications, effectively facilitating low-latency data transmission. Our experimental evaluations across diverse traces of wireless scenarios confirm that Colla enhances throughput, latency, and tail latency. Looking ahead, we aim to expand the impact of our research by incorporating our findings into rate control modules used in protocols such as QUIC and WebRTC. This integration will better cater to the multifaceted service requirements of real-time applications.

## ACKNOWLEDGEMENT

This work was supported by the National Natural Science Foundation of China (92167204, 62072030, 92267301), and the National Key Research and Development Program of China (2023YFB2904205).

## REFERENCES

- [1] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM computer communication review*, vol. 18, no. 4, pp. 314–329, 1988.
- [2] S. Ha, I. Rhee, and L. Xu, "Cubic: a new tcp-friendly high-speed tcp variant," *ACM SIGOPS operating systems review*, vol. 42, no. 5, pp. 64–74, 2008.
- [3] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, no. 5, pp. 20–53, 2016.
- [4] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center tcp (dctcp)," in *Proceedings of the ACM SIGCOMM 2010 Conference*, 2010, pp. 63–74.
- [5] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "{PCC}: Re-architecting congestion control for consistent high performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015, pp. 395–408.
- [6] K. Winstein, A. Sivaraman, and H. Balakrishnan, "Stochastic forecasts achieve high throughput and low delay over cellular networks," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013, pp. 459–471.
- [7] Y. Zaki, T. Pötsch, J. Chen, L. Subramanian, and C. Görg, "Adaptive congestion control for unpredictable cellular networks," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, 2015, pp. 509–522.
- [8] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the internet," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, 2020, pp. 632–647.
- [9] A. Narayan, F. Cangialosi, D. Raghavan, P. Goyal, S. Narayana, R. Mittal, M. Alizadeh, and H. Balakrishnan, "Restructuring endpoint congestion control," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 30–43.
- [10] Z. Meng, J. Chen, Y. Guo, C. Sun, H. Hu, and M. Xu, "Pitree: Practical implementation of abr algorithms using decision trees," in *Proceedings of the 27th ACM International Conference on Multimedia*, 2019, pp. 2431–2439.
- [11] G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo, "Analysis and design of the google congestion control for web real-time communication (webrrc)," in *Proceedings of the 7th International Conference on Multimedia Systems*, 2016, pp. 1–12.
- [12] A. Langley, A. Riddoch, A. Wilk, A. Vicente, C. Krasic, D. Zhang, F. Yang, F. Kouranov, I. Swett, J. Iyengar *et al.*, "The quic transport protocol: Design and internet-scale deployment," in *Proceedings of the conference of the ACM special interest group on data communication*, 2017, pp. 183–196.
- [13] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "{PCC} vivace:{Online-Learning} congestion control," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 343–356.
- [14] A. Kleen, H. M. Khosravi, A. Kuznetsov, and J. H. Salim, "Linux Netlink as an IP Services Protocol," RFC 3549, Jul. 2003. [Online]. Available: <https://www.rfc-editor.org/info/rfc3549>
- [15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*, PMLR, 2016, pp. 1928–1937.
- [16] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: the training ground for internet congestion-control research," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 731–743.
- [17] R. Netravali, A. Sivaraman, S. Das, A. Goyal, K. Winstein, J. Mickens, and H. Balakrishnan, "Mahimahi: accurate {Record-and-Replay} for {HTTP}," in *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, 2015, pp. 417–429.
- [18] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Wanna make your tcp scheme great for cellular networks? let machines do it for you!" *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 265–279, 2020.
- [19] V. Arun and H. Balakrishnan, "Copa: Practical {Delay-Based} congestion control for the internet," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 329–342.
- [20] S. Floyd, T. Henderson, and A. Gurtov, "The newreno modification to tcp's fast recovery algorithm," Tech. Rep., 2004.
- [21] L. Xu, K. Harfoush, and I. Rhee, "Binary increase congestion control (bic) for fast long-distance networks," in *IEEE INFOCOM 2004*, vol. 4, IEEE, 2004, pp. 2514–2524.
- [22] L. S. Brakmo, S. W. O'malley, and L. L. Peterson, "Tcp vegas: New techniques for congestion detection and avoidance," in *Proceedings of the conference on Communications architectures, protocols and applications*, 1994, pp. 24–35.
- [23] K. Tan, J. Song, Q. Zhang, and M. Sridharan, "A compound tcp approach for high-speed and long distance networks," in *Proceedings-IEEE INFOCOM*, 2006.
- [24] S. Liu, T. Başar, and R. Srikant, "Tcp-illinois: A loss and delay-based congestion control algorithm for high-speed networks," in *Proceedings of the 1st international conference on Performance evaluation methodologies and tools*, 2006, pp. 55–es.
- [25] S. Floyd, D. K. K. Ramakrishnan, and D. L. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," RFC 3168, Sep. 2001. [Online]. Available: <https://www.rfc-editor.org/info/rfc3168>
- [26] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion control for large-scale rdma deployments," *ACM SIGCOMM Computer Communication Review*, vol. 45, no. 4, pp. 523–536, 2015.
- [27] S. Abbasloo, Y. Xu, and H. J. Chao, "C2tcp: A flexible cellular tcp to meet stringent delay requirements," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 4, pp. 918–932, 2019.
- [28] K. Winstein and H. Balakrishnan, "Tcp ex machina: Computer-generated congestion control," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 123–134, 2013.
- [29] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "Qtcp: Adaptive congestion control with reinforcement learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2018.
- [30] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on internet congestion control," in *International Conference on Machine Learning*. PMLR, 2019, pp. 3050–3059.